EXP.NO: 5	CPU Schedulling
DATE: 13/02/2026	

AIM:

To implement different CPO scheduling calgorithm (FCFS, SJF, Priority cand Round Robin) cand evaluate their performance by calculating Average Jurnaround Time and Average Maiting time.

ALGORITHM:

1. nitialization: Create a structure/class for Processes containing PID, AT, BT, and Priority.
2. Time Tracking: Maintain a global currentTime variable.
3. The Selection Loop: While processes remain:
4. Identify all processes that have arrived (AT ≤ currentTime).
5. Apply the specific Selection Criteria (e.g., Min BT, Max Priority).
6. Update the CPU clock and mark completion.
7. Calculation: Compute averages for TAT and WT across all processes.

CODE:

FCFS

```
#include <stdio.h>
int main() {
   int n, ct = 0;
   float tat = 0, wt = 0;
   printf("Enter number of processes:\n");
   scanf("%d", &n);

   for (int i = 0, at, bt; i < n; i++) {
      printf("Enter AT and BT for P%d:\n", i + 1);
      scanf("%d %d", &at, &bt);

      if (ct < at) ct = at;
      ct += bt;

      tat += (ct - at);
      wt += (ct - at - bt);
   }
```

```
    printf("\nAvg TAT = %.2f\nAvg WT = %.2f\n", tat/n, wt/n);
    return 0;
}
```

SJF

```
#include <stdio.h>

int main() {
    int n, ct[20], tat[20], wt[20], at[20], bt[20], done[20] = {0};
    printf("Enter number of processes:\n");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter AT and BT for P%d:\n", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
    }

    int time = 0, completed = 0;

    while (completed < n) {
        int idx = -1;
        for (int i = 0; i < n; i++) {
            if (!done[i] && at[i] <= time &&
               (idx == -1 || bt[i] < bt[idx]))
                idx = i;
        }

        if (idx == -1) { time++; continue; }

        time += bt[idx];
        ct[idx] = time;
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];

        done[idx] = 1;
        completed++;
    }
```

```
    float avg_tat = 0, avg_wt = 0;
    for (int i = 0; i < n; i++) {
        avg_tat += tat[i];
        avg_wt += wt[i];
    }

    printf("\nAvg TAT = %.2f\nAvg WT = %.2f\n", avg_tat / n, avg_wt / n);
    return 0;
}
```

PS

```
#include <stdio.h>

int main() {
    int n, t = 0, c = 0, idx;
    float tw = 0, tt = 0;

    scanf("%d", &n);
    int at[n], bt[n], pr[n], d[n];

    for (int i = 0; i < n; i++) {
        scanf("%d%d%d", &at[i], &bt[i], &pr[i]);
        d[i] = 0;
    }

    while (c < n) {
        int best = 9999;
        idx = -1;

        for (int i = 0; i < n; i++)
            if (!d[i] && at[i] <= t && pr[i] < best)
                best = pr[i], idx = i;

        if (idx == -1) { t++; continue; }

        tw += t - at[idx];
        t += bt[idx];
        tt += t - at[idx];
```

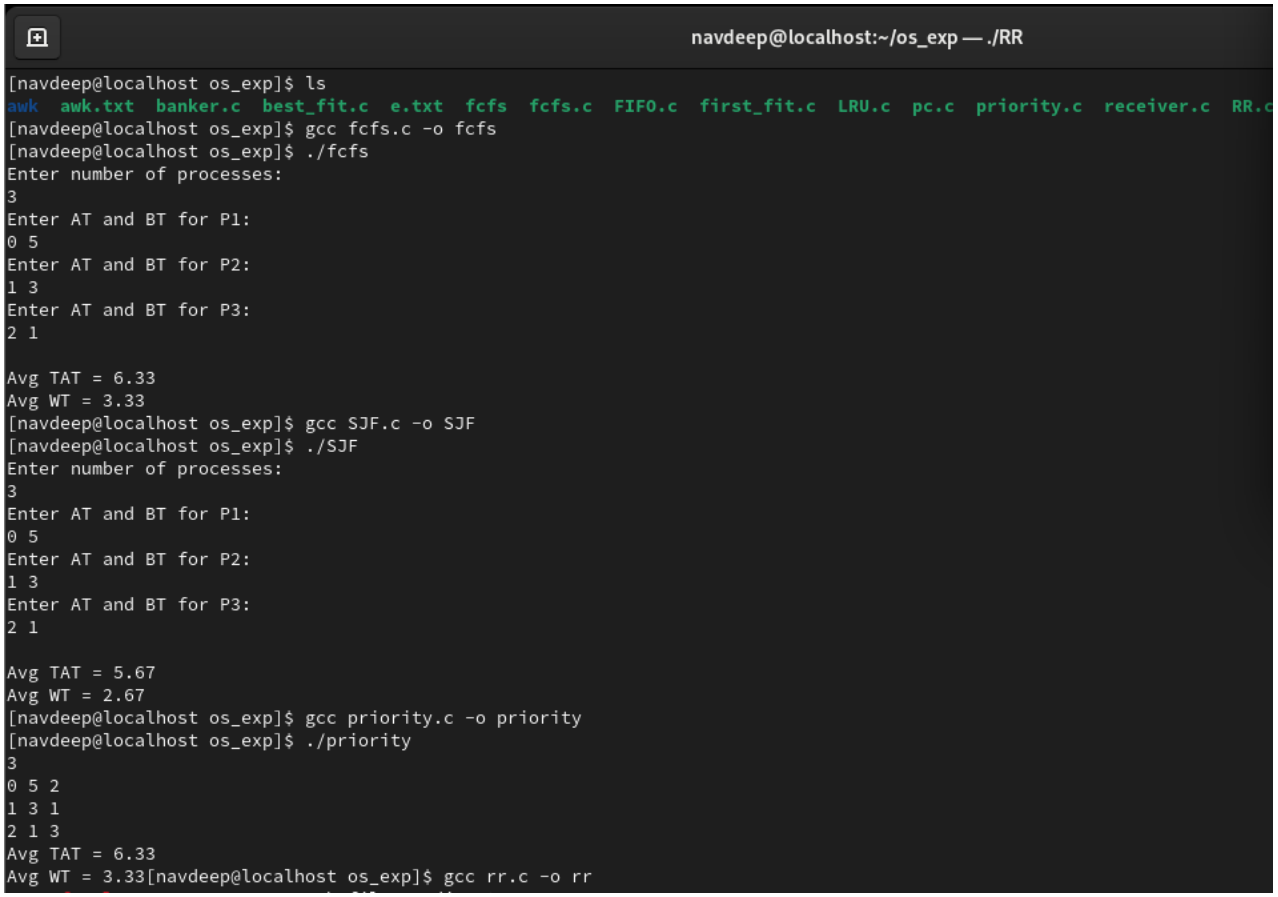
```
        d[idx] = 1;
        c++;
    }

    printf("Avg TAT = %.2f\nAvg WT = %.2f", tt/n, tw/n);
}
```

RR

```
#include <stdio.h>
int main() {
    int n, tq, time = 0, done = 0;
    scanf("%d %d", &n, &tq);

    int at[n], bt[n], rem[n], ct[n];
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &at[i], &bt[i]);
        rem[i] = bt[i];
    }

    while (done < n) {
        int flag = 0;
        for (int i = 0; i < n; i++) {
            if (rem[i] > 0 && at[i] <= time) {
                flag = 1;
                if (rem[i] <= tq) {
                    time += rem[i];
                    ct[i] = time;
                    rem[i] = 0;
                    done++;
                } else {
                    time += tq;
                    rem[i] -= tq;
                }
            }
        }
        if (!flag) time++;
    }
```

```
    float tat = 0, wt = 0;
    for (int i = 0; i < n; i++) {
        tat += ct[i] - at[i];
        wt += ct[i] - at[i] - bt[i];
    }

    printf("Avg TAT=%.2f\nAvg WT=%.2f", tat/n, wt/n);
}
```

OUTPUT:



RESULT:

The implementation confirms that **SJF** is the mathematically "optimal" algorithm for minimizing wait times. However, in modern operating systems, **Round Robin** or a **Multilevel Feedback Queue** is preferred because they prevent "starvation" (where a long process never gets to run) and provide better responsiveness for the user.